

CompareTool 使用说明书

代码比对报告工具 Windows 桌面版

本说明书帮助使用者选择比较方式、生成差异报告、查看变更文件，并核对交付目录。适用于 Git、SVN、文件夹、压缩包、Git多版本和 SVN多版本六种模式。

先记住：旧版本表示改动前，新版本表示改动后。工具生成 HTML 报告和变更文件；导出目录保存完整的变更文件内容，不包含全部未变更文件。

第一次使用

1. 双击 CompareTool.exe。只有两个本地文件夹时，在顶部“版本控制类型”选择“文件夹”。
2. 分别选择旧文件夹和新文件夹，确认项目名，检查“排除规则”。
3. 选择独立的输出目录，确认“输出批次名称”，点击“生成比对报告”。
4. 核对弹窗中的实际输出目录，继续生成；完成后打开报告，并检查同目录的上线操作说明和 oldVersion、newVersion。

阅读导航

操作主题	页码
启动条件与界面字段	2
两个文件夹比对	3
两个压缩包比对	4
Git 与 SVN 双版本比对	5
Git 与 SVN 多版本比对	6
多项目总报告	7
排除规则与显示选项	8
阅读 HTML 差异报告	9
导出目录与交付核对	10
配置保存与日常使用	11
常见问题处理	12

目录可点击跳转；也可使用 PDF 阅读器的书签面板。本文中的 Demo、版本号和 D 盘示例路径均用于说明操作，请替换为实际项目。

02 启动条件与界面字段

运行前准备

将 CompareTool.exe 放在允许写入配置和日志的目录，双击启动。使用打包版无需安装 Python；程序可能同时带有控制台窗口，这是正常现象。

比较方式	需要准备的内容
文件夹 / 压缩包	两个可读取的本地文件夹，或两个受支持的压缩包；这两种模式不依赖 Git 或 SVN。
Git / Git多版本	本地 Git 仓库，以及可用的 Git 命令行工具；相关提交对象需要在本地存在。
SVN / SVN多版本	项目工作副本、SVN 命令行工具，以及访问仓库所需的网络和读取权限。

程序会自动查找 Git / SVN 命令行工具。若使用 TortoiseSVN，安装时需包含 command line client tools。已有凭据或网络问题时，先确认该仓库能被命令行正常访问。

界面主要字段

界面字段	如何填写或使用
版本控制类型	选择六种比较方式之一。文件夹入口已经在这里，无需先打包。
项目目录	Git / SVN 类模式选择仓库或项目目录；文件夹和压缩包模式会隐藏此项。
项目名	用于报告、清单和导出子目录。自动推断后仍应核对；无法推断时手工填写真实名称。
排除规则	每行一条路径匹配规则。命中的文件不参加本次比较和导出。
旧版本与新版本	根据模式填写版本号或选择来源路径；多版本模式的新侧显示“文件级首尾端点”。
输出目录与批次名称	输出目录是基础位置；填写批次名时再增加一层子目录。生成前会提示实际位置。
显示选项	控制报告路径是否显示项目名，以及展示全部内容还是差异上下文。
多项目批量任务	将多个项目的配置分别加入列表，然后生成统一报告和导出目录。

路径建议：输出目录与输入目录分开，例如输入位于 D:\compare_demo\old 和 new，输出放到 D:\compare_output。不要把导出目录放进待比较的文件夹。

03 两个文件夹比对

适合比较两份源码目录、两个部署目录，或手工解压后的两个目录。比较依据是所选根目录下的相对文件路径和内容。

操作步骤

1. 在“版本控制类型”选择“文件夹”。界面出现“旧版本文件夹”和“新版本文件夹”。
2. 点击两侧“浏览...”选择目录，或在输入框中填入完整路径。旧侧放改动前目录，新侧放改动后目录。
3. 核对项目名。若新目录叫 release_new，但项目实际名称是 Demo，可将项目名改为 Demo。
4. 检查排除规则，特别是部署内容中的 .class、bin、build 等是否需要比较。
5. 设置输出目录和批次名称，选择差异展示方式，点击“生成比对报告”。
6. 确认实际输出目录，等待“完成”提示；打开报告检查结果。

选择同一层级的比较根目录

旧侧根目录	新侧根目录	报告中的相对路径
D:\demo_old	D:\demo_new	config/app.xml
D:\demo_old\app	D:\demo_new\app	config/app.xml

如果旧侧选到 demo_old，而新侧误选到 demo_new\app，两侧层级可能不一致，原本对应的文件会出现大量新增、删除或重命名。应先确认两边从同一业务层级开始比较。

一个简单示例

文件变化	预期报告	导出位置
新增 config/new.xml	新增	只在 newVersion 中出现
修改 config/app.xml	修改或格式变化	oldVersion 和 newVersion 各有一份
删除 config/old.xml	删除	只在 oldVersion 中出现，并写入上线操作说明

使用时注意：生成期间避免编辑、构建或替换输入目录。程序会固定待比较的文件状态；若检测到来源变化，可能中止并提示重新生成。选择同一个文件夹作为两端，正常结果为零变更。

本模式用于文件差异，不应将结果理解为目录权限、所有空目录等全部文件系统属性的完整审计。

04 两个压缩包比对

适合比较两个发布包、两个源码归档或两个 Web 应用包。工具先展开两端压缩包，再比较内部文件；不需要手工解压到项目目录。

支持的格式

格式家族	可选择的扩展名
ZIP 及同类归档	.zip、.jar、.war、.ear、.aar
TAR 及压缩 TAR	.tar、.tar.gz、.tgz、.tar.bz2、.tbz2

两侧可以是不同的受支持归档格式，例如 ZIP 对 TAR。当前没有 RAR / 7z 输入支持，也没有文件夹与压缩包直接混合比较入口。需要混合比较时，先手工解压压缩包，再使用“文件夹”模式。

操作步骤

1. 选择“压缩包”，分别点击“选择压缩包...”设置旧版本和新版本。
2. 核对项目名及排除规则。比较 WAR 内的编译结果时，按需要移除 *.class 等排除规则。
3. 设置输出目录、批次名称和显示选项，点击“生成比对报告”。
4. 确认实际输出位置并等待完成。报告显示包内文件差异，导出目录保存发生变化的内部文件。

忽略最外层单一文件夹

默认保留包内完整路径。若两包都只有一个顶层文件夹，可勾选“忽略最外层单一文件夹”：
release_old/config/app.xml 与 release_new/config/app.xml 会按 config/app.xml 比较。只去掉一层；报告、排除规则和导出使用内层路径，报告与上线说明注明两侧比较根。

任一包顶层有文件、多个文件夹，或顶层为空时会报错。可取消勾选，或手工解压后选择目录进行文件夹比对。此选项按新包路径记忆，多项目任务也会保存。

外层压缩包与内部压缩文件

作为输入的压缩包会展开；包内的 lib/dependency.jar 等归档通常作为二进制文件比较，不会自动继续逐层展开。它们有变化时仍会完整导出。导出结果也不会自动重新打包成 WAR、JAR 或 ZIP。

压缩包无法处理时

压缩包损坏、包含危险路径、符号链接或特殊文件时会失败。现有解压边界为最多 100,000 个成员、单成员 2 GiB、累计展开 10 GiB、压缩比 1000:1；超过边界时不能按成功结果交付。磁盘空间也需满足实际展开和输出需求。

05 Git 与 SVN 双版本比对

普通 Git / SVN 模式比较两个指定版本的项目状态。适合检查上次发布版本到本次发布版本之间的完整净变化。

模式	旧版本与新版本的输入形式	示例
Git	提交哈希、标签或分支	旧 v1.0; 新 v1.1
SVN	revision 数字，可带 r 前缀	旧 r100; 新 r108

操作步骤

1. 选择 “Git” 或 “SVN” ，再选择对应的本地项目目录。Git 可选仓库根或项目子目录；SVN 选择对应项目的工作副本目录。
2. 核对项目名和排除规则。最近项目下拉可同时回填项目目录与项目名。
3. 在旧版本一侧点击 “获取版本列表” ，选中目标版本，点击 “← 填入选中版本” ，或双击版本记录。
4. 在新版本一侧重复操作。注意列表上方 “将填入” 指向旧版本还是新版本，防止填错一侧。也可以直接输入明确的版本号。
5. 设置输出目录、批次名称及显示选项，生成报告并核对结果。

版本列表怎么用

普通 Git 列表包含标签、分支和最近 100 条提交日志；普通 SVN 显示最近 100 条 revision。搜索框只筛选当前已加载的列表，不会搜索整个仓库历史。较早的版本不在列表中时，可直接输入已确认的有效版本号。

“清空” 清除搜索词；“隐藏版本列表” 只收起列表，点击 “显示版本列表” 可以恢复。切换项目目录或比较类型后，版本输入和列表会重置，应重新选择。

需要理解的比较范围

Git 选择子目录时，报告和导出以该子目录为项目范围。普通 Git / SVN 使用所选仓库版本；本地尚未提交的文件修改不会自动作为新版本参与比较。若要比较两份磁盘上的实际文件，使用文件夹模式。

Git 仓库可变标签或分支会在任务开始时固定到实际提交；SVN 会固定本次访问的仓库身份和 revision。为便于日后复核，重要交付优先记录明确的提交哈希或 revision。

06 Git 与 SVN 多版本比对

当一次需求涉及多个提交或 revision，希望集中查看相关文件的最终变化时，选择“Git多版本”或“SVN多版本”。

操作步骤

1. 选择多版本模式和项目目录，核对项目名与排除规则。
2. 点击“获取版本列表”，用 Ctrl 选择不连续记录，用 Shift 选择连续范围，再点击“← 填入选中版本”。也可手工用逗号、分号或换行分隔版本号。
3. 确认版本输入内容。“生成结果”固定为“文件级首尾端点”，无需填写一个统一的新版本号。
4. 设置输出目录后生成。搜索过滤和隐藏列表会保留多选状态；更换项目或重新获取列表后应重新核对选择。

文件级首尾端点是什么意思

对每个被选中版本改动过的文件，旧侧取该文件第一次选中变更之前的状态，新侧取该文件最后一次选中变更之后的状态。每个文件分别确定自己的首尾版本，只展示最终净差异。

示例中的选中变更	oldVersion 取值	newVersion 取值
A.xml 在 r3 和 r6 被修改	A.xml 在 r2 的状态	A.xml 在 r6 的状态
B.xml 只在 r3 被修改	B.xml 在 r2 的状态	B.xml 在 r3 的状态
C.xml 只在 r6 被修改	C.xml 在 r5 的状态	C.xml 在 r6 的状态

上述示例用 SVN revision 表示。Git 对应取首次选中提交的第一父提交，以及末次选中提交。新增后删除、修改后恢复等最终抵消的文件，不进入报告和导出。

选中提交不等于只抽出这些提交的代码片段

重要：如果 A.xml 在未选中的 r4 也被修改，而选中的 r6 仍保留那部分内容，A.xml 的新侧就会包含它。工具不会执行 cherry-pick 或 SVN merge，也不会剔除历史文件中已经存在的中间修改。

newVersion 中每个文件都是完整文件，但不同文件可能来自不同版本，因此它不是某个统一版本的完整项目快照。交付前应确认这些文件状态符合需求。

Git 多版本只接受当前分支第一父历史中的提交；列表中的 [merge] 表示合并提交。缺少必要历史对象、最终文件为特殊节点或无法唯一确认文件身份时，任务会中止。

07 多项目总报告

多项目任务适合一次交付多个服务或模块。每个任务按自己的模式生成结果；不同任务可以使用不同的 Git、SVN、文件夹、压缩包或多版本模式。

添加和生成

1. 先在主界面配置第一个项目，填写来源和版本，确认项目名、排除规则和显示选项。
2. 点击“添加到多项目任务”，确认列表中出现该项目。
3. 继续配置其他项目并逐个添加。不同项目必须使用不同名称，Demo 和 demo 也按重名处理。
4. 核对当前全局“输出目录”和“输出批次名称”，点击“生成多项目总报告”。
5. 确认本次实际输出目录。全部任务成功后，统一生成报告、说明和两套变更文件；任何一个任务失败，本次生成失败。

修改已添加的任务

选中任务后点击“编辑任务”，设置会回填到主界面；修改后点击“更新多项目任务”保存到列表。“取消编辑”退出编辑状态。“删除任务”和“清空任务”操作的是任务列表，不是删除源项目或已经生成的文件。

每个任务保存添加或更新时的设置。后来修改主界面的排除规则或显示选项，不会自动改掉已添加的任务；需要编辑对应任务后更新。输出批次名称是本次生成的全局设置，不随单个任务保存。

多项目输出放在哪里

```
D:\compare_output\20260908\  
multi_run_20260908_143000_123_a1b2c3d4\  
multi_compare_report_20260908_143000_123.html  
上线操作说明.txt  
oldVersion\  
ServiceA\  
ServiceB\  
newVersion\  
ServiceA\  
ServiceB\  

```

每次多项目生成都会创建新的 multi_run 运行目录，时间和随机后缀用于区分不同结果。移交时保留同一运行目录中的完整文件，不要混用不同运行目录中的报告和源码包。

同名展示路径：若两个项目都含 config/app.xml，且都关闭项目名显示，报告路径可能冲突。将相关任务的“报告树及变更清单使用项目名”设为“是”后更新任务，再重新生成。

08 排除规则与显示选项

排除规则会影响比较和导出

排除规则每行一条，按项目内的相对路径匹配。任何一条规则命中，就会排除该文件。建议统一使用正斜杠 /；留空表示没有额外的路径排除规则，读取与文件安全检查仍然生效。

规则示例	含义
*.class	排除任意深度的 .class 文件。
target/**	排除项目根目录下 target 目录中的内容。
/target/	排除任意位置 target 目录中的内容。
config/*.tmp	排除 config 目录直接包含的 .tmp 文件。
/.git/	排除任意位置 .git 目录中的内容。

单个 * 匹配单层路径中的字符，** 可跨越目录层级。按界面支持的 * 和 ** 规则填写，不要套用 Git 的忽略文件语法；不要依赖 ! 反向包含或注释行。

比较部署包时先调整默认规则

默认规则偏向源码比较，会排除 *.class、*.war、*.ear、target、build、bin、dist、日志、临时目录和常见开发工具元数据。它们在部署目录中可能恰好就是需要检查的内容。

例如要比较 WEB-INF/classes 下的 .class 文件，就移除 *.class；要比较实际 bin 目录中的程序或脚本，就移除 **/bin/**。修改后再生成，并核对目标文件是否出现在变更结果中。当前界面没有“部署内容比较”预设，需要手工调整。

排除规则按项目来源记忆：Git / SVN 类按项目目录，文件夹按新文件夹，压缩包按新压缩包的完整路径。换一个来源后，应重新检查实际显示的规则。

两项显示设置

设置	效果
报告树及变更清单使用项目名	“是” 显示 Demo/config/app.xml；“否” 显示 config/app.xml。此项不取消导出目录中的项目名，也不改变多项目上线说明中的项目名前缀。
全部内容	报告展示文本文件的完整内容，便于逐行检查。
仅差异上下文	报告展示变更附近前后各 3 行，适合快速审阅。导出的文件仍是完整文件。

09 阅读 HTML 差异报告

生成完成后可选择立即打开报告，也可在输出目录双击 .html 文件。报告是单文件 HTML，可用浏览器查看；交付源码时还应一并保留同批次的导出目录和上线操作说明。

推荐阅读顺序

1. 先看报告头部的项目、来源和版本信息，确认没有把新旧方向或项目选错。
2. 查看新增、修改、格式变化、删除和重命名的汇总数量。行数统计与文件数量是不同指标，二进制文件通常没有逐行内容。
3. 在左侧文件树选择文件，点击目录展开或折叠，也可使用“展开全部 / 折叠全部”。
4. 在右侧对照旧内容和新内容，用“上一处 / 下一处”跳转当前文件的差异。长行可横向滚动；鼠标点击定位后，可用左右方向键横向查看。
5. 点击“查看变更清单”，在新窗口中查看按类型分类的完整路径清单；需要时可选择文字复制。

五类变更标识

标识	含义	阅读重点
A 新增	旧侧没有，新侧存在	检查新增路径和完整文件内容。
M 修改	同一文件有内容等变化	查看具体差异及文件说明。
F 格式变化	正文等价但编码、BOM 或换行等发生变化	仍是实际文件变化，需要核对并导出。
D 删除	旧侧存在，新侧没有	结合上线操作说明确认旧路径。
R 重命名	旧路径变为新路径	核对旧路径、新路径及伴随的内容变化。

类型筛选标签只显示本次实际存在的类型，可组合筛选。“全部”恢复全部类型。隐藏某一类型仅影响当前报告的查看范围，不会重新生成或删除已导出的文件。

看不到逐行内容或遇到特殊标记

JAR、WAR、CLASS、图片等二进制文件会显示占位说明；未被排除且发生变化的文件仍完整导出。未知编码字节会显示十六进制标记（如 0xFF），特殊分隔字符会显示字符编号（如 U+2028）。这些是报告中的展示标记，导出文件保留其真实字节或字符。

10 导出目录与交付核对

单项目输出示例

假设项目名为 Demo，输出目录为 D:\compare_output，批次名称为 20260908，生成位置如下。批次名称留空时，直接输出到基础输出目录。

```
D:\compare_output\20260908\  
Demo_diff.html  
Demo_上线操作说明.txt  
oldVersion\  
  Demo\config\app.xml  
newVersion\  
  Demo\config\app.xml
```

哪些文件在哪一侧

变更类型	oldVersion	newVersion
新增	没有该文件	新增后的完整文件
修改或格式变化	改动前的完整文件	改动后的完整文件
删除	删除前的完整文件	没有该文件
重命名	旧路径下的完整文件	新路径下的完整文件
未变化或净变化抵消	不导出	不导出

导出范围：这是变更文件集合，不是整个项目的备份，也不是代码补丁片段。多版本导出还可能包含来自不同提交的文件。仅差异上下文选项不会截断导出的文件。

上线操作说明的作用

说明文件列出需要删除的旧路径、旧目录替换要求以及重命名关系。没有相关操作时，也会生成说明并明确标注。单纯复制 newVersion 不会自动删除目标环境里的旧文件，必须结合说明按项目发布流程处理。

Git / SVN 导出的换行符按所选版本的属性和配置处理；文件夹和压缩包保留对应文件字节。工具生成结果不代表已完成编译、部署或业务验收。

交付前核对

1. 确认报告的项目名、来源和版本与本次发布一致，重点抽查关键文件。
2. 确认报告、上线操作说明、oldVersion 和 newVersion 属于同一次生成，新增与删除所在的一侧正确。
3. 保存整套输出。单项目在相同批次、相同项目名下重复生成会替换同名报告、说明和该项目导出；需保留历史时使用不同批次名。多项目则保留对应的独立 multi_run 目录。

11 配置保存与日常使用

哪些设置会记住

内容	保存行为
项目路径和比较类型	下次启动时恢复上次保存的设置。
最近项目	Git / SVN 分家族保存最近 10 个有效项目；可从项目名下拉选择。
排除规则和显示选项	按项目来源分别记忆；新项目没有专属配置时使用默认规则。
多项目任务	保存任务中的来源、版本及当时的排除和显示设置。
输出目录	保存上次设置的基础输出目录。
输出批次名称	不持久化；每次启动默认系统当天日期。需要发布批次名称时重新填写。

打包版配置文件名为 `compareTool_config.json`，与 `CompareTool.exe` 放在同一目录。普通双版本的版本输入在重新选择项目或切换模式时会清空；每次生成前都应核对两端，不把记忆的界面设置当作已确认的发布版本。

最近项目与多项目任务的区别

最近项目帮助快速回填项目目录和项目名，不是一条已经确认版本的批量任务。切换最近项目会清空当前版本选择。多项目任务保存的是添加或更新时的一整套配置，生成时逐项执行。

保存失败或配置损坏

配置写入失败会提示“配置保存失败”，新任务可能只保留在当前会话。先检查程序所在目录的写入权限及文件是否被占用，再处理保存问题，避免直接关闭后丢失新设置。

配置文件本身无法解析时，程序可用默认设置启动，但会保留损坏文件并阻止覆盖。先备份 `compareTool_config.json`，再修复；如果确定不再恢复旧设置，可在程序关闭后将原文件改名保留，再重新启动配置。

临时文件和日志

比较时会产生专用临时内容，正常成功或失败后会清理。文件较多或压缩包较大时，保持输入、临时位置和输出所在磁盘有足够可用空间。程序没有按预计耗时给正常生成设置固定超时，不能仅凭等待较久判断结果已失败。

警告和错误记录在程序同目录的 `compareTool.log`；日志可能轮转为 `compareTool.log.bak`。正常过程通常不会持续写日志，因此日志没有更新不等于程序没有工作。

生成期间关闭主窗口时，程序会提示等待任务完成。遇到异常先保留错误提示、日志和相关输出，不要在生成或恢复过程中手工删除临时文件或事务文件。

12 常见问题处理

找不到文件夹比对入口

看顶部“版本控制类型”，选择“文件夹”。随后两侧输入会变为“旧版本文件夹 / 新版本文件夹”。如果正在运行的界面没有该项，核对是否打开了其他位置的旧 EXE。

版本列表搜不到目标提交

搜索只过滤当前已加载列表，通常只含最近 100 条日志。确认项目和模式正确后，直接输入已知有效版本号。Git 多版本还要求提交位于当前分支第一父历史；仓库缺历史对象时需先补齐。

生成后没有变化或少了预期文件

检查新旧路径和版本、目录层级及排除规则。两包只有外层目录名不同，可使用第 4 页的“忽略最外层单一文件夹”。Git / SVN 普通模式不比较未提交修改；多版本的最终净变化抵消也不会列出。

压缩包与文件夹不能放在两侧

当前模式要求两侧都是文件夹，或两侧都是受支持压缩包。先手工解压压缩包，选择对应的内容根目录，再用文件夹模式比较。RAR / 7z 也可先借助已有解压工具解压。

报错找不到 Git 或 SVN 或无法读取仓库

确认已安装命令行工具，重新启动程序后重试。仓库访问失败时检查网络、地址、账号权限和认证状态。SVN 工作副本必须指向准备比较的项目；不要仅根据文件夹名称判断仓库身份。

提示项目重名或报告展示路径冲突

项目名按 Windows 规则判重，不区分大小写，应改用不同名称。路径展示冲突时，编辑任务并将“报告树及变更清单使用项目名”设为“是”，更新任务后重试。

提示来源变化 路径冲突 或转换不支持

停止对输入目录的编辑或构建，待来源稳定后重试。路径碰撞、链接或特殊文件以及不可复现的 Git / SVN 转换可能使任务失败；按错误指出的具体文件检查，不能把不完整结果当成成功输出。

报告未自动打开或变更清单没有弹出

先确认界面已提示完成，到实际输出目录手工打开 .html。变更清单通过新窗口展示，若浏览器拦截弹窗，在允许本地报告弹窗后重试。不要仅因浏览器没有打开就重复生成覆盖已有结果。

需要反馈问题时记录什么

记录比较模式、项目范围、两端版本或路径、排除规则、实际输出目录和完整错误提示；附程序同目录的 compareTool.log。能够提供最小的脱敏复现目录或压缩包时，更便于定位。